

如何在 Cloud Run 部署安全的 Genkit MCP 伺服器

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/genkit-how-to-deploy-a-secure-mcp-server-on-cloud-run?hl=zh-tw>

步驟數：12

瞭解如何在 Cloud Run 上建構及部署安全的 Genkit Model Context Protocol (MCP) 伺服器。接著向遠端 MCP 伺服器進行驗證，並從 Gemini CLI 叫用 MCP 工具和提示。

目錄

1. [1. 簡介](#)
2. [2. 專案設定](#)
3. [3. 開啟 Cloud Shell 編輯器](#)
4. [4. 啟用 API](#)
5. [5. 準備 JavaScript 專案](#)
6. [6. 建立動物園 MCP 伺服器](#)
7. [7. 部署至 Cloud Run](#)
8. [8. 將遠端 MCP 伺服器新增至 Gemini CLI](#)
9. [9. \(選用\) 驗證伺服器記錄中的工具呼叫](#)
10. [10. \(選用\) 在伺服器中新增 MCP 提示](#)
11. [11. \(選用\) 使用 Gemini Flash Lite 快速取得回覆](#)
12. [12. 結論](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

在本實驗室中，您將建構及部署 Model Context Protocol (MCP) 伺服器。MCP 伺服器可讓大型語言模型存取外部工具和服務。您會在 Cloud Run 上將其設定為安全且可供正式環境使用的服務，並可從多個用戶端存取。接著，您將從 Gemini CLI 連線至遠端 MCP 伺服器。

學習內容

我們將使用 [Genkit](#) 建立 **zoo MCP 伺服器**，其中包含 `get_animals_by_species` 和 `get_animal_details` 這兩項工具。Genkit 提供快速建構 MCP 伺服器和 Node.js 用戶端的方法。



課程內容

- 將 MCP 伺服器部署至 Cloud Run。
 - 為伺服器端點設定所有要求的驗證機制，確保只有授權的用戶端和代理程式可以與其通訊，藉此保護端點安全。
 - 從 Gemini CLI 連線至安全的 MCP 伺服器端點
-

2. 專案設定

1. 如果沒有 Google 帳戶，請先[建立帳戶](#)。
 - 請改用個人帳戶，而非公司或學校帳戶。公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。
 2. 登入 [Google Cloud 控制台](#)。
 3. 在 Cloud 控制台中[啟用帳單](#)。
 - 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
 - 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
 - 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。
 4. [建立新專案](#)，或選擇重複使用現有專案。
 - 如果看到專案配額相關錯誤，請重複使用現有專案，或刪除現有專案來建立新專案。
-

步驟 3 · 約 0 分鐘

3. 開啟 Cloud Shell 編輯器

1. 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

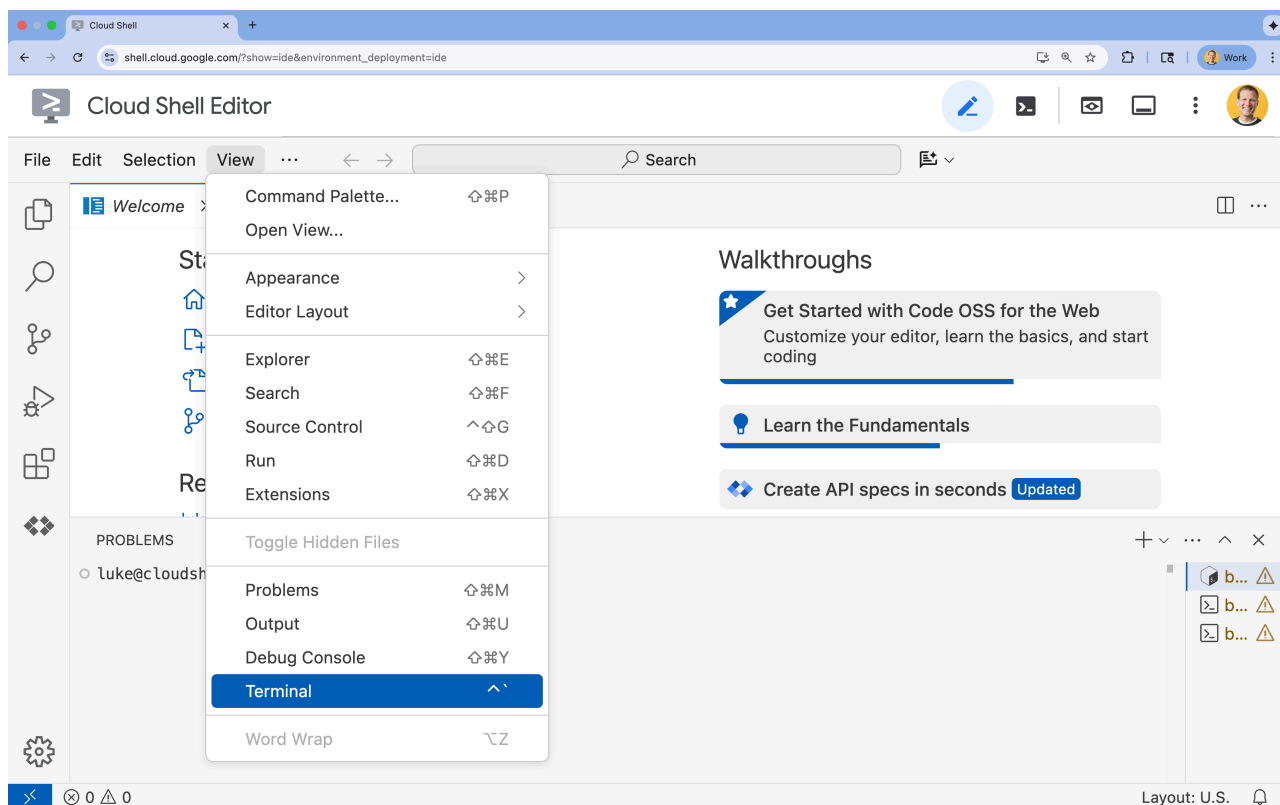
Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下「查看」
- 按一下「終端機」



4. 在終端機中，使用下列指令設定專案：

- 格式：

```
gcloud config set project [PROJECT_ID]
```

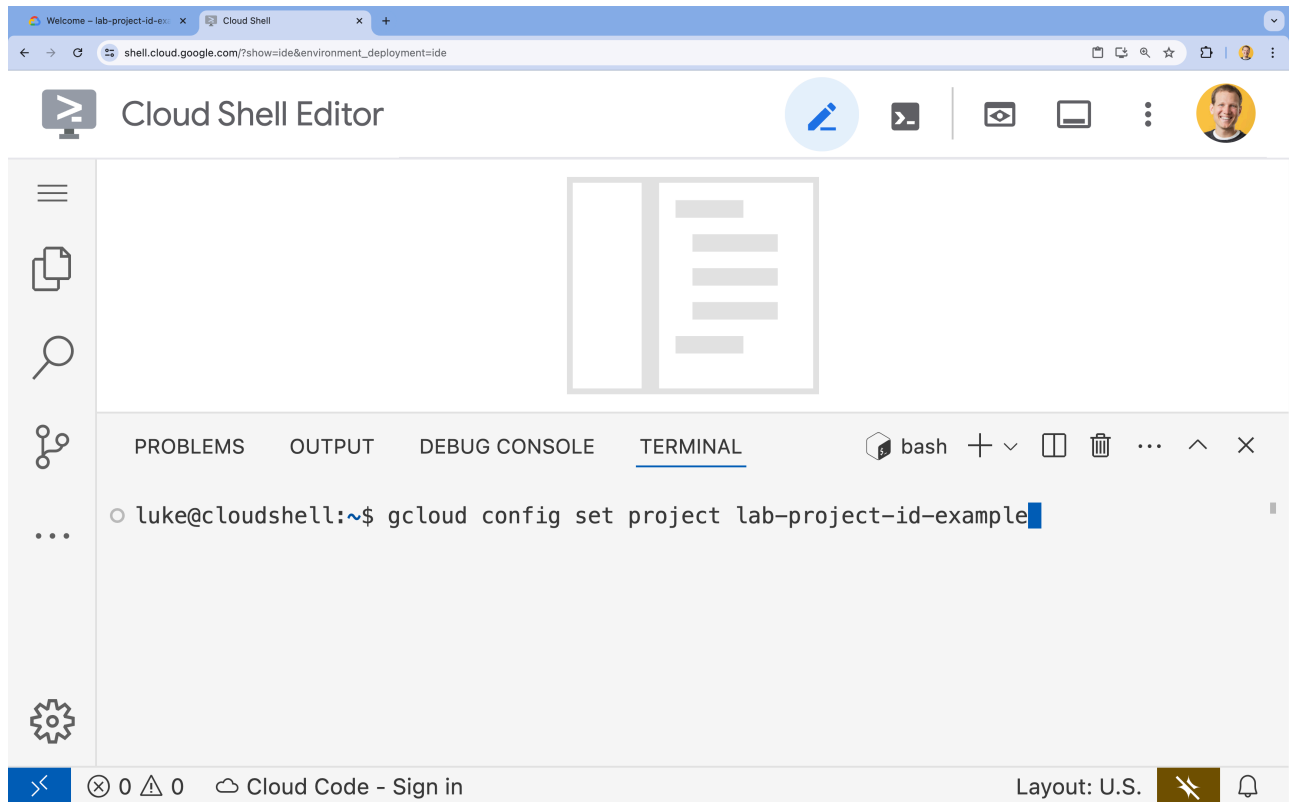
- 範例：

```
gcloud config set project lab-project-id-example
```

◦ 如果忘記專案 ID，請按照下列步驟操作：

- 您可以使用下列指令列出所有專案 ID：

```
gcloud projects list | awk '/PROJECT_ID/{print $2}'
```



5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 `WARNING` 並收到 `Do you want to continue (Y/n)?` 提示，可能是專案 ID 輸入有誤。按下 `n` 和 `Enter`，然後再次嘗試執行 `gcloud config set project` 指令。

步驟 4 · 約 0 分鐘

4. 啟用 API

在終端機中啟用 API：

```
gcloud services enable \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com
```

如果系統提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

這個指令可能需要幾分鐘才能完成，但最終應該會產生類似以下的成功訊息：

```
Operation "operations/acf.p2-73d90d00-47ee-447a-b600" finished successfully.
```

步驟 5 · 約 0 分鐘

5. 準備 JavaScript 專案

1. 建立名為 `mcp-on-cloudrun` 的資料夾，用來儲存部署作業的原始碼：

```
mkdir mcp-on-cloudrun && cd mcp-on-cloudrun
```

2. 使用 `npm` 工具建立 Node.js 專案，產生 `package.json` 檔案：

```
npm init es6 -y
```

`npm init` 指令會為專案建立 `package.json` 檔案。

3. 安裝 `@modelcontextprotocol/sdk`、`express` 和 `zod` 依附元件：

```
npm install @modelcontextprotocol/sdk express zod
```

6. 建立動物園 MCP 伺服器

如要提供有價值的脈絡，以利改善搭配 MCP 使用 LLM 的體驗，請使用 [Genkit](#) 設定動物園 MCP 伺服器。Genkit 是使用 Model Context Protocol 的標準框架。Genkit 提供快速建構 MCP 伺服器和 Node.js 用戶端的方法。這個 MCP 伺服器提供虛構動物園的動物資料。為簡化操作，我們將資料儲存在記憶體中。如果是正式版 MCP 伺服器，您可能會想從資料庫或 API 等來源提供資料。

1. 執行下列指令，將 Genkit 新增為 `package.json` 檔案中的依附元件：

```
npm install genkit
```

這會在專案中新增 `package-lock.json` 檔案。

2. 執行下列指令，在 `package.json` 檔案中新增 Genkit AI MCP 程式庫：

```
npm install @genkit-ai/mcp
```

3. 建立並開啟新的 `index.js` 檔案，用於 MCP 伺服器原始碼：

```
cloudshell edit index.js
```

`cloudshell edit` 指令會在終端機上方的編輯器中開啟 `index.js` 檔案。

4. 在 `index.js` 檔案中新增下列 zoo MCP 伺服器原始碼：

```

import express from 'express';
import { genkit, z } from 'genkit';
import { createMcpServer } from '@genkit-ai/mcp';
import { StreamableHTTPServerTransport } from
 '@modelcontextprotocol/sdk/server/streamableHttp.js';

// Dictionary of animals at the zoo
const ZOO_ANIMALS = [
  {
    "species": "lion",
    "name": "Leo",
    "age": 7,
    "enclosure": "The Big Cat Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "lion",
    "name": "Nala",
    "age": 6,
    "enclosure": "The Big Cat Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "lion",
    "name": "Simba",
    "age": 3,
    "enclosure": "The Big Cat Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "lion",
    "name": "King",
    "age": 8,
    "enclosure": "The Big Cat Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "penguin",
    "name": "Waddles",
    "age": 2,
    "enclosure": "The Arctic Exhibit",
    "trail": "Polar Path"
  },
  {
    "species": "penguin",
    "name": "Pip",
    "age": 4,
    "enclosure": "The Arctic Exhibit",
    "trail": "Polar Path"
  },
  {
    "species": "penguin",

```

```
    "name": "Skipper",
    "age": 5,
    "enclosure": "The Arctic Exhibit",
    "trail": "Polar Path"
  },
  {
    "species": "penguin",
    "name": "Chilly",
    "age": 3,
    "enclosure": "The Arctic Exhibit",
    "trail": "Polar Path"
  },
  {
    "species": "penguin",
    "name": "Pingu",
    "age": 6,
    "enclosure": "The Arctic Exhibit",
    "trail": "Polar Path"
  },
  {
    "species": "penguin",
    "name": "Noot",
    "age": 1,
    "enclosure": "The Arctic Exhibit",
    "trail": "Polar Path"
  },
  {
    "species": "elephant",
    "name": "Ellie",
    "age": 15,
    "enclosure": "The Pachyderm Sanctuary",
    "trail": "Savannah Heights"
  },
  {
    "species": "elephant",
    "name": "Peanut",
    "age": 12,
    "enclosure": "The Pachyderm Sanctuary",
    "trail": "Savannah Heights"
  },
  {
    "species": "elephant",
    "name": "Dumbo",
    "age": 5,
    "enclosure": "The Pachyderm Sanctuary",
    "trail": "Savannah Heights"
  },
  {
    "species": "elephant",
    "name": "Trunkers",
    "age": 10,
    "enclosure": "The Pachyderm Sanctuary",
```

```
    "trail": "Savannah Heights"
  },
  {
    "species": "bear",
    "name": "Smokey",
    "age": 10,
    "enclosure": "The Grizzly Gulch",
    "trail": "Polar Path"
  },
  {
    "species": "bear",
    "name": "Grizzly",
    "age": 8,
    "enclosure": "The Grizzly Gulch",
    "trail": "Polar Path"
  },
  {
    "species": "bear",
    "name": "Barnaby",
    "age": 6,
    "enclosure": "The Grizzly Gulch",
    "trail": "Polar Path"
  },
  {
    "species": "bear",
    "name": "Bruin",
    "age": 12,
    "enclosure": "The Grizzly Gulch",
    "trail": "Polar Path"
  },
  {
    "species": "giraffe",
    "name": "Gerald",
    "age": 4,
    "enclosure": "The Tall Grass Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "giraffe",
    "name": "Longneck",
    "age": 5,
    "enclosure": "The Tall Grass Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "giraffe",
    "name": "Patches",
    "age": 3,
    "enclosure": "The Tall Grass Plains",
    "trail": "Savannah Heights"
  },
  {
```

```
    "species": "giraffe",
    "name": "Stretch",
    "age": 6,
    "enclosure": "The Tall Grass Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "antelope",
    "name": "Speedy",
    "age": 2,
    "enclosure": "The Tall Grass Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "antelope",
    "name": "Dash",
    "age": 3,
    "enclosure": "The Tall Grass Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "antelope",
    "name": "Gazelle",
    "age": 4,
    "enclosure": "The Tall Grass Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "antelope",
    "name": "Swift",
    "age": 5,
    "enclosure": "The Tall Grass Plains",
    "trail": "Savannah Heights"
  },
  {
    "species": "polar bear",
    "name": "Snowflake",
    "age": 7,
    "enclosure": "The Arctic Exhibit",
    "trail": "Polar Path"
  },
  {
    "species": "polar bear",
    "name": "Blizzard",
    "age": 5,
    "enclosure": "The Arctic Exhibit",
    "trail": "Polar Path"
  },
  {
    "species": "polar bear",
    "name": "Iceberg",
    "age": 9,
```

```

        "enclosure": "The Arctic Exhibit",
        "trail": "Polar Path"
    },
    {
        "species": "walrus",
        "name": "Wally",
        "age": 10,
        "enclosure": "The Walrus Cove",
        "trail": "Polar Path"
    },
    {
        "species": "walrus",
        "name": "Tusker",
        "age": 12,
        "enclosure": "The Walrus Cove",
        "trail": "Polar Path"
    },
    {
        "species": "walrus",
        "name": "Moby",
        "age": 8,
        "enclosure": "The Walrus Cove",
        "trail": "Polar Path"
    },
    {
        "species": "walrus",
        "name": "Flippers",
        "age": 9,
        "enclosure": "The Walrus Cove",
        "trail": "Polar Path"
    }
];

// Initialize Genkit
const ai = genkit({});

// Define tools using Genkit
ai.defineTool(
    {
        name: 'get_animals_by_species',
        description: "Retrieves all animals of a specific species from the zoo. Can also be used to collect the base data for aggregate queries of animals of a specific species - like counting the number of penguins or finding the oldest lion.",
        inputSchema: z.object({ species: z.string() }),
    },
    async ({ species }) => {
        console.log(`>>> 🦾 Tool: 'get_animals_by_species' called for '${species}'`);
        const animals = ZOO_ANIMALS.filter(animal => animal.species.toLowerCase() === species.toLowerCase());
        return animals;
    }
);

```

```

ai.defineTool(
  {
    name: 'get_animal_details',
    description: "Retrieves the details of a specific animal by its name.",
    inputSchema: z.object({ name: z.string() }),
  },
  async ({ name }) => {
    console.log(`>>> 🛠️ Tool: 'get_animal_details' called for '${name}'`);
    const animal = ZOO_ANIMALS.find(a => a.name.toLowerCase() ===
name.toLowerCase());
    return animal;
  }
);

// Create Genkit MCP server wrapper
const mcpWrapper = createMcpServer(ai, {
  name: 'zoo-animal-server',
  version: '1.0.0',
});

// HTTP server mode
const app = express();
app.use(express.json());

// Initialize Genkit MCP server once
const mcpServerPromise = mcpWrapper.setup().then(() => mcpWrapper.server);

app.post('/mcp', async (req, res) => {
  console.log('/mcp Received:', req.body);
  console.log('Using HTTP transport mode.');
```

```

  const server = await mcpServerPromise;

  const transport = new StreamableHTTPServerTransport({
    sessionIdGenerator: undefined,
  });

  if (!server) {
    console.error('MCP Server not initialized correctly.');
```

```

    res.sendStatus(500);
    return;
  }

  await server.connect(transport);
  await transport.handleRequest(req, res, req.body);

  res.on('close', () => {
    console.log('Request closed');
```

```

    transport.close();
  });
});

```

```
app.get('/mcp', async (req, res) => {
  console.log('Received GET MCP request');
  res.sendStatus(405);
});

app.delete('/mcp', async (req, res) => {
  console.log('Received DELETE MCP request');
  res.sendStatus(405);
});

// Start the server
const PORT = process.env.PORT || 8080;
app.listen(PORT, () => {
  console.log(`Zoo Animal MCP server listening on port ${PORT}`);
});
```

程式碼已完成！接下來，請將 MCP 伺服器部署至 Cloud Run。

7. 部署至 Cloud Run

現在直接從原始碼將 MCP 伺服器部署至 Cloud Run。

1. 建立名為 `mcp-server-sa` 的服務帳戶：

```
gcloud iam service-accounts create mcp-server-sa --display-name="MCP Server Service Account"
```

2. 執行 `gcloud` 指令，將應用程式部署至 Cloud Run

```
gcloud run deploy zoo-mcp-server \
  --service-account=mcp-server-sa@$GOOGLE_CLOUD_PROJECT.iam.gserviceaccount.com \
  --no-allow-unauthenticated \
  --region=europe-west4 \
  --source=. \
  --labels=dev-tutorial=codelab-mcp
```

使用 `--no-allow-unauthenticated` 標記來要求驗證。這是出於安全考量。如果不需要驗證，任何人都可以呼叫 MCP 伺服器，可能導致系統損壞。

3. 確認已建立新的 Artifact Registry 存放區。由於您是第一次從原始碼部署至 Cloud Run，因此會看到：

```
Deploying from source requires an Artifact Registry Docker repository to store
built containers. A repository named
[cloud-run-source-deploy] in region [europe-west4] will be created.

Do you want to continue (Y/n)?
```

輸入 `y` 並按下 `Enter`，系統會為部署作業建立 Artifact Registry 存放區。這是儲存 Cloud Run 服務的 MCP 伺服器 Docker 容器時的必要步驟。

4. 幾分鐘後，您會看到類似下列的訊息：

```
Service [zoo-mcp-server] revision [zoo-mcp-server-12345-abc] has been deployed and
is serving 100 percent of traffic.
```

注意：如果您嘗試直接造訪網址，會看到「Error Forbidden」，因為 MCP 伺服器需要驗證。

您已部署 MCP 伺服器。現在可以使用了。

8. 將遠端 MCP 伺服器新增至 Gemini CLI

您已成功部署遠端 MCP 伺服器，現在可以使用 Google Code Assist 或 Gemini CLI 等各種應用程式連線至該伺服器。在本節中，我們將使用 [Gemini CLI](#) 建立與新遠端 MCP 伺服器的連線。

注意：Cloud Shell 已預先安裝 Gemini CLI。

1. 授予使用者帳戶呼叫遠端 MCP 伺服器的權限

```
gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT \
  --member=user:$(gcloud config get-value account) \
  --role='roles/run.invoker'
```

2. 將 Google Cloud 憑證和專案編號儲存至環境變數，供 Gemini 設定檔使用：

```
export PROJECT_NUMBER=$(gcloud projects describe $GOOGLE_CLOUD_PROJECT --
format="value(projectNumber)")
export ID_TOKEN=$(gcloud auth print-identity-token)
```

3. 開啟 Gemini CLI 設定檔

```
cloudshell edit ~/.gemini/settings.json
```

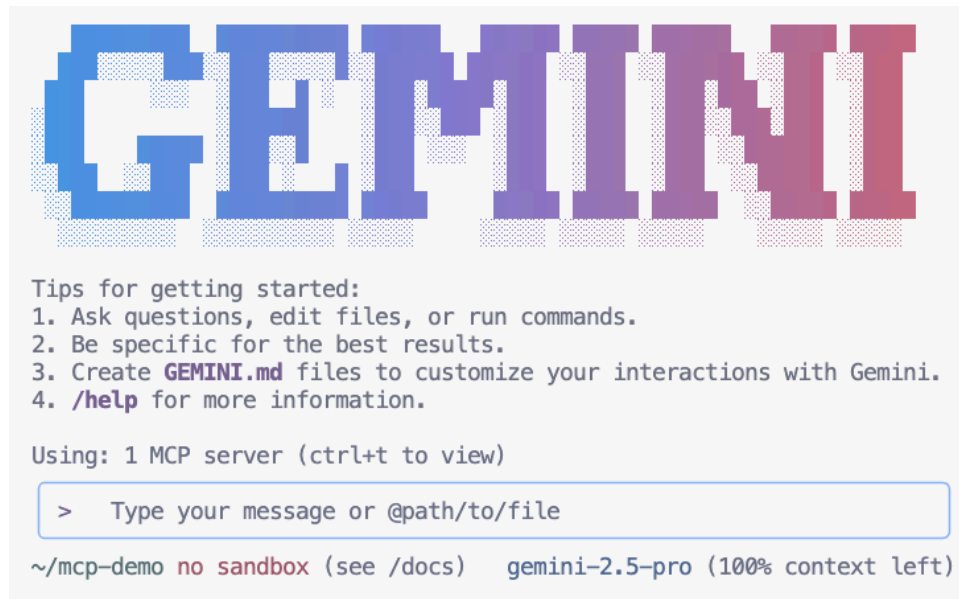
4. 取代 Gemini CLI 設定檔，新增 Cloud Run MCP 伺服器

```
{
  "ide": {
    "hasSeenNudge": true
  },
  "mcpServers": {
    "zoo-remote": {
      "httpUrl": "https://zoo-mcp-server-$PROJECT_NUMBER.europe-west4.run.app/mcp",
      "headers": {
        "Authorization": "Bearer $ID_TOKEN"
      }
    }
  },
  "security": {
    "auth": {
      "selectedType": "cloud-shell"
    }
  }
}
```

1. 在 Cloud Shell 中啟動 Gemini CLI

```
gemini
```

您可能需要按下 `Enter` 鍵來接受部分預設設定。



2. 讓 Gemini 列出在目前環境中可用的 MCP 工具

```
/mcp
```

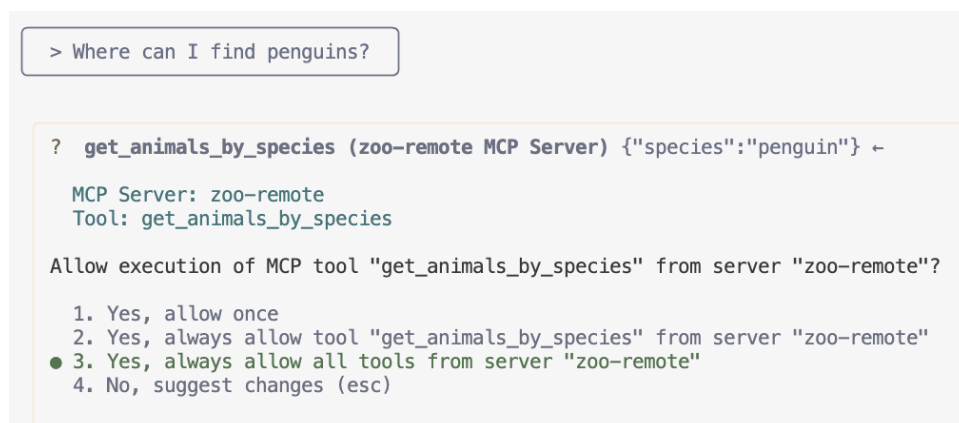
3. 請 Gemini 找出動物園中的某種動物

```
Where can I find penguins?
```

Gemini CLI 會判斷需使用 `zoo-remote` MCP 伺服器，並詢問您是否允許執行 MCP。

4. 使用向下鍵，然後按下 `Enter` 鍵選取

```
Yes, always allow all tools from server "zoo-remote"
```



輸出內容應會顯示正確答案，以及表示已使用 MCP 伺服器的顯示方塊。

```
> Where can I find penguins?

✓ get_animals_by_species (zoo-remote MCP Server) {"species":"penguin"}

[{"species":"penguin","name":"Waddles","age":2,"enclosure":"The Arctic Exhibit","trail":"Polar Path"}, {"species":"penguin","name":"Pip","age":4,"enclosure":"The Arctic Exhibit","trail":"Polar Path"}, {"species":"penguin","name":"Skipper","age":5,"enclosure":"The Arctic Exhibit","trail":"Polar Path"}, {"species":"penguin","name":"Chilly","age":3,"enclosure":"The Arctic Exhibit","trail":"Polar Path"}, {"species":"penguin","name":"Pingu","age":6,"enclosure":"The Arctic Exhibit","trail":"Polar Path"}, {"species":"penguin","name":"Noot","age":1,"enclosure":"The Arctic Exhibit","trail":"Polar Path"}]

+ You can find the penguins in the Arctic Exhibit on the Polar Path.
```

你做到了！您已成功將遠端 MCP 伺服器部署至 Cloud Run，並使用 Gemini CLI 進行測試。

如要結束工作階段，請輸入 `/quit`，然後按下 `Enter` 退出 Gemini CLI。

偵錯

如果看到類似下方的錯誤訊息：

```
🔍 Attempting OAuth discovery for 'zoo-remote'...
❌ 'zoo-remote' requires authentication but no OAuth configuration found
Error connecting to MCP server 'zoo-remote': MCP server 'zoo-remote' requires authentication. Please configure OAuth or check server settings.
```

ID 權杖可能已逾時，需要再次設定 `ID_TOKEN`。

1. 輸入 `/quit`，然後按下 `Enter` 鍵，即可退出 Gemini CLI。
2. 在終端機中設定專案

```
gcloud config set project [PROJECT_ID]
```

3. 重新執行 Google Cloud 憑證指令，取得新的 `ID_TOKEN`，因為您目前的憑證可能已過期。

```
export PROJECT_NUMBER=$(gcloud projects describe $GOOGLE_CLOUD_PROJECT --
format="value(projectNumber)")
export ID_TOKEN=$(gcloud auth print-identity-token)
```

步驟 9 · 約 0 分鐘

9. (選用) 驗證伺服器記錄中的工具呼叫

如要確認 Cloud Run MCP 伺服器是否已呼叫，請查看服務記錄。

```
gcloud run services logs read zoo-mcp-server --region europe-west4 --limit=5
```

系統應會傳回確認已呼叫工具的輸出記錄。

```
2025-08-05 19:50:31 INFO:      169.254.169.126:39444 - "POST /mcp/ HTTP/1.1" 200 OK
2025-08-05 19:50:31 [INFO]: Processing request of type CallToolRequest
2025-08-05 19:50:31 [INFO]: >>>  Tool: 'get_animals_by_species' called for
'penguin'
```

10. (選用) 在伺服器中新增 MCP 提示

對於經常執行的提示，您可以建立較長提示的簡短版本，藉此加快工作流程。

Gemini CLI 會自動將 MCP 提示詞轉換為[自訂斜線指令](#)，因此只要輸入 `/prompt_name`，即可叫用 MCP 提示詞，其中 `prompt_name` 是 MCP 提示詞的名稱。

建立 MCP 提示詞，在 Gemini CLI 中輸入 `/find animal` 即可快速找到動物園中的動物。

1. 在 `index.js` 檔案的 `// Create Genkit MCP server wrapper` 行上方加入下列程式碼：

```
ai.definePrompt(
  {
    name: 'find',
    description: 'Find which exhibit and trail a specific animal is located.',
    inputSchema: z.object({ animal: z.string() }),
  },
  async ({ animal }) => {
    console.log(`>>> 🗨️ Prompt: 'find' called`);
    return {
      messages: [
        {
          role: 'user',
          content: [
            { text: `Please find the exhibit and trail information for ${animal} in the zoo. Respond with '[animal] can be found in the [exhibit] on the [trail].' Example: Penguins can be found in The Arctic Exhibit on the Polar Path.` }
          ]
        }
      ]
    };
  }
);
```

2. 將應用程式重新部署至 Cloud Run

```
gcloud run deploy zoo-mcp-server \
  --service-account=mcp-server-sa@$GOOGLE_CLOUD_PROJECT.iam.gserviceaccount.com \
  --no-allow-unauthenticated \
  --region=europe-west4 \
  --source=. \
  --labels=dev-tutorial=codelab-mcp
```

3. 更新遠端 MCP 伺服器的 ID_TOKEN

```
export ID_TOKEN=$(gcloud auth print-identity-token)
```

4. 部署新版應用程式後，請啟動 Gemini CLI。

gemini

5. 在提示中使用您建立的新自訂指令：

```
/find --animal="lions"
```

Gemini CLI 應會呼叫 `get_animals_by_species` 工具，並按照 MCP 提示詞的指示格式化回覆！

```
> /find --animal="lion" |
```

```
| ✓ get_animals_by_species (zoo-remote MCP Server) get_animals_by_species (zoo-remote MCP Server) |
|
| [{"species":"lion","name":"Leo","age":7,"enclosure":"The Big Cat
| Plains","trail":"Savannah
| Heights"}, {"species":"lion","name":"Nala","age":6,"enclosure":"The Big Cat
| Plains","trail":"Savannah
| Heights"}, {"species":"lion","name":"Simba","age":3,"enclosure":"The Big Cat
| Plains","trail":"Savannah
| Heights"}, {"species":"lion","name":"King","age":8,"enclosure":"The Big Cat
| Plains","trail":"Savannah Heights"}]
```

✦ Lions can be found in The Big Cat Plains on the Savannah Heights.

11. (選用) 使用 Gemini Flash Lite 快速取得回覆

Gemini CLI 可讓您選擇要使用的模型。

- **Gemini 2.5 Pro** 是 Google 最先進的思考型模型，能夠針對程式碼、數學和 STEM 領域的複雜問題進行推論，並運用長脈絡分析大型資料集、程式碼集和文件。
- **Gemini 2.5 Flash** 是 Google 成本效益最高的模型，功能十分全面。2.5 Flash 最適合用於大規模處理、低延遲、需要思考的大量工作，以及代理程式應用實例。
- **Gemini 2.5 Flash Lite** 是 Google 速度最快的 Flash 模型，經過最佳化處理，成本效益高且高處理量。

由於尋找動物園動物的要求不需要思考或推論，因此請嘗試使用速度較快的模型，加快處理速度。

建立 MCP 提示詞，在 Gemini CLI 中輸入 `/find animal` 即可快速找到動物園中的動物。

1. 部署新版應用程式後，請啟動 Gemini CLI。

```
gemini --model=gemini-2.5-flash-lite
```

2. 在提示中使用您建立的新自訂指令：

```
/find --animal="lions"
```

您應該還是會看到 Gemini CLI 呼叫 `get_animals_by_species` 工具，並按照 MCP 提示詞的指示格式化回覆，但答案應該會快很多！

```
> /find --animal="lion" |
```

```
| ✓ get_animals_by_species (zoo-remote MCP Server) get_animals_by_species (zoo-remote MCP Server) |
|
| [{"species":"lion","name":"Leo","age":7,"enclosure":"The Big Cat
| Plains","trail":"Savannah
| Heights"}, {"species":"lion","name":"Nala","age":6,"enclosure":"The Big Cat
| Plains","trail":"Savannah
| Heights"}, {"species":"lion","name":"Simba","age":3,"enclosure":"The Big Cat
| Plains","trail":"Savannah
| Heights"}, {"species":"lion","name":"King","age":8,"enclosure":"The Big Cat
| Plains","trail":"Savannah Heights"}]
```

✦ Lions can be found in The Big Cat Plains on the Savannah Heights.

挑戰目標，測試自己的實力

如要增加挑戰難度，請嘗試按照相同步驟建立提示詞，傳回動物園中特定動物物種的有趣事實。

或者，您也可以進一步測試所學內容，構思您經常使用的工具，並部署第二個遠端 MCP 伺服器。然後將其新增至 Gemini CLI 設定，確認是否正常運作。

偵錯

如果看到類似下方的錯誤訊息：

```
✕ Unknown command: /find --animal="lions"
```

請嘗試執行 `/mcp`，如果輸出 `zoo-remote - Disconnected`，您可能需要重新部署，或再次執行下列指令：

```
gcloud projects add-iam-policy-binding $GOOGLE_CLOUD_PROJECT \
  --member=user:${(gcloud config get-value account)} \
  --role='roles/run.invoker'

export PROJECT_NUMBER=$(gcloud projects describe $GOOGLE_CLOUD_PROJECT --
format="value(projectNumber)")
export ID_TOKEN=$(gcloud auth print-identity-token)
```

12. 結論

恭喜！您已成功部署並連線至安全的遠端 MCP 伺服器。

繼續下一個實驗室

本實驗室是三部分系列的第一個實驗室。在第二個實驗室中，您將使用透過 ADK 代理建立的 MCP 伺服器。

[搭配 ADK 代理使用 Cloud Run 上的 MCP 伺服器](#)

(選用) 清理

如果您不會繼續進行下一個實驗室，且想清理您建立的內容，可以刪除雲端專案，以免產生額外費用。

不使用服務時，Cloud Run 不會收費，但您可能仍須支付容器映像檔在 Artifact Registry 的儲存費用。刪除雲端專案後，系統就會停止對專案使用的所有資源收取費用。

如要刪除專案，請按照下列步驟操作：

```
gcloud projects delete $GOOGLE_CLOUD_PROJECT
```

您也可以從 Cloud Shell 磁碟刪除不必要的資源。您可以：

1. 刪除 Codelab 專案目錄：

```
rm -rf ~/mcp-on-cloudrun
```

2. 警告！這項操作無法復原，如要刪除 Cloud Shell 中的所有內容來釋出空間，可以[刪除整個主目錄](#)。請務必將要保留的內容另存他處。

```
sudo rm -rf $HOME
```
